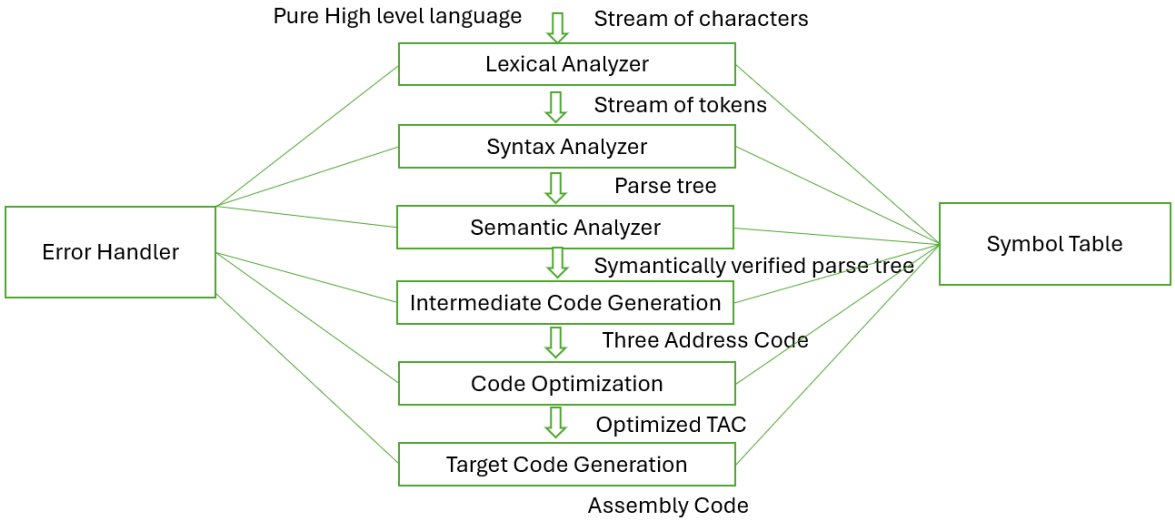
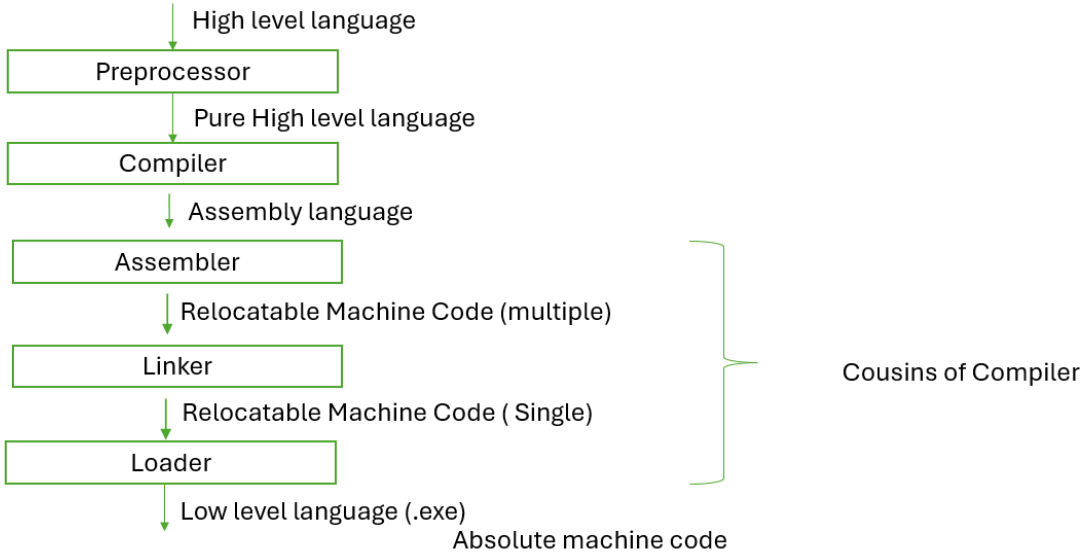
MID PART

6 Phases of Compiler



LPS (Language Processing System)



GROUP_C

How to check given Grammar is LL(1)

- The parsing table of a grammar may contain more than one production rule i.e. multiple defined. This is possible if G is left recursive or ambiguous.
- In this case, we say that it is not a LL(1) grammar.
- A grammar whose parsing table has no multiply-defined entries is said to be LL(1) grammar.

How to check given Grammar is LL(1)

- If G doesn't contain ε

  $A \rightarrow \alpha_1 \mid \alpha_2 \mid \alpha_3$

  $First(\alpha_1) \cap First(\alpha_2) = \emptyset$

  $First(\alpha_2) \cap First(\alpha_3) = \emptyset$

  $First(\alpha_3) \cap First(\alpha_1) = \emptyset$

- If G contains ε

  $A \rightarrow \alpha_1 \mid \alpha_2 \mid \varepsilon$

  $First(\alpha_1) \cap First(\alpha_2) = \emptyset$

  $First(\alpha_1) \cap Follow(A) = \emptyset$

  $First(\alpha_2) \cap Follow(A) = \emptyset$

LL(1) Parser – Parser Actions

- The symbol at the top of the stack (say X) and the current symbol in the input string (say a) determine the parser action.
- There are four possible parser actions.
  1. If (X = $, a = $) parser halts (successful completion)
  2. If X= a (but not $) parser pops X from the stack, and moves the next symbol in the input buffer.
  3. If X is a non-terminal
     - Parser looks at the parsing table entry M[X, a].
     - If M[X, a] holds a production rule $X \rightarrow Y_1 Y_2 \ldots Y_k$, it pops X from the stack and pushes $Y_k, Y_{k-1}, \ldots, Y_1$ into the stack.
     - The parser also outputs the production rule $X \rightarrow Y_1 Y_2 \ldots Y_k$ to represent a step of the derivation.

LL(1) Parser – Parser Actions

4. None of the above : error
   - All empty entries in the parsing table are errors.
   - If X is a terminal symbol different from a, this is also an error case.

PARSER ACTION

LL(1) Parser – What Actions It Can Take

The parser always looks at two things:

The symbol on top of the stack (call it X).

The current symbol in the input (call it a).

Based on these, the parser decides what to do.

There are 4 possible actions:

1. Successful Finish

If X = $ (end of stack) and a = $ (end of input),
→ Parsing is done successfully.

2. Match Terminal

If X is a terminal symbol and it is the same as a (but not $),
→ Remove X from the stack and move to the next symbol in the input.

3. Expand Non-terminal

If X is a non-terminal:

Look up the parsing table at M[X, a].

If there's a rule like X → Y1 Y2 … Yk, then:

Pop X from the stack.

Push the symbols Yk … Y2 Y1 into the stack (revers order so Y1 is on top).

Also record/output this production rule as a step i the derivation.

4. Error
If none of the above applies, it's an error.
Errors happen when:
The parsing table entry is empty.

The top of stack is a terminal symbol but it doesn't match the input symbol.

Ex2:

- $S \rightarrow aSA \mid \varepsilon$
  $A \rightarrow c \mid \varepsilon$

Check if this grammar is LL(1) or not

Solution:

- For this grammar $S \rightarrow aSA \mid \varepsilon$
  - First(aSA) = {a} , Follow(S) = {$,c}, First(aSA) ∩ Follow(S) = ∅
- For this grammar $A \rightarrow c \mid \varepsilon$
  - First(c) = {c} , Follow(A) = {$,c}, First(c) ∩ Follow(A) = {c}
- So, this grammar is not LL(1)

EX3:

- $S \rightarrow aSbS \mid bSaS \mid \varepsilon$

Check if this grammar is LL(1) or not

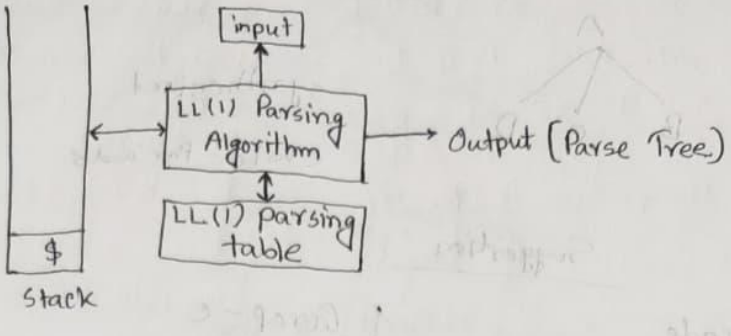
Solution:

For this grammar $S \rightarrow aSbS \mid bSaS \mid \varepsilon$

- First(aSbS) = {a} , First (bSaS) = {b}, Follow(S) = {$, b, a}
- First(aSbS) ∩ First (bSaS) = ∅
- First(aSbS) ∩ Follow(S) = {a}
- First (bSaS) ∩ Follow(S) = {b}

- So, this grammar is not LL(1)

LL(1) Parser



① No ambiguity
② no left recursion
③ First & follow
④ Parsing Table
⑤ Stack Implementation
⑥ Parse Tree.

Ex: $E \to TE'$
$E' \to \epsilon \mid +TE'$
$T \to FT'$
$T' \to \epsilon \mid *FT'$
$F \to id \mid (E)$

First and follow:

Grammer	First	Follow
E	{id, (}	{$,)}
E'	{ϵ, +}	{$,)}
T	{id, (}	{+, $,)}
T'	{ϵ, *}	{+, $,)}
F	{id, (}	{*, +, $,)}

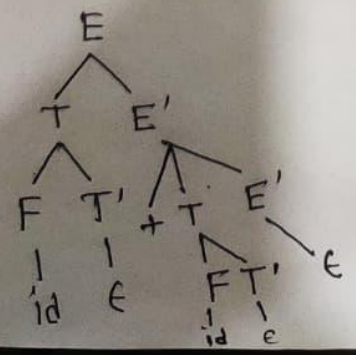
Parsing Table:

M	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to id$			$F \to (E)$		

Stack Implementation :

Stack	input	Output
$ E	id+id$	E → TE'
$ E'T	id+id$	T → FT'
$ E'T'F	id+id$	F → id
$ E'T'id	id+id$	pop
$ E'T'	+id$	T' → ϵ
$ E'	+id$	E' → +TE'
$ E'T+	+id$	pop
$ E'T	id$	T → FT'
$ E'T'F	id$	F → id
$ E'T'id	id$	pop
$ E'T'	$	T' → ϵ
$ E'	$	E' → ϵ
$	$	accept.

Parse Tree



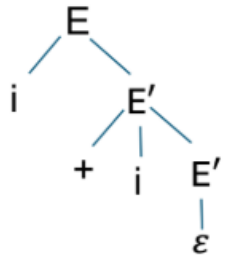
Recursive Descent Parser

- It is a top down parser.
- Non-deterministic grammar, Left recursive grammar and ambiguous grammar is not allowed here.
- Right recursive grammar is allowed here.

How Recursive Descent Parser works?

E-> i E′, E′ -> +i E′ | ε

Let's take a string i+i.



We will write functions for corresponding variables. We will use i+i$ ($ represents the end of input or the end of a string)

How Recursive Descent Parser works?

```
E′()
{
  if(input==‘+’)
    {  input++;
       if(input==‘i’)
        { input++;
          E′(); }
     }
else
    {return; }
```

```
E ()
{
  If (input==‘i’)
     {input++;
     E′();}
}
```

```
Main()
{
   E();
   if( input ==‘$’)
        {printf(“Parsing
successful”);}
}
```

Recursion

A→ $A\alpha \mid \beta$

Eliminating left recursion

A -> βA'

A' -> $\varepsilon \mid \alpha$A'

It is like a formulla

EX1

E -> E+T | T

Now

E -> E+T | T

A A $\alpha \mid \beta$

eliminating left recursion:

E-> T E'

E'-> ε | +TE'

Recursion

$A \rightarrow A\alpha \mid \beta$

Eliminating left recursion

$A \rightarrow \beta A'$

$A' \rightarrow \varepsilon \mid \alpha A'$

It is like a formulla

Ex2:

S -> S01S0S | 01

Now,

S -> S 01S0S | 01

A A α | β

eliminating left recursion:

S -> 01S'

S' -> ε | 01S0S S'

Left Factoring

- Converting nondeterministic grammar to deterministic grammar is called left factoring.

ND =>
Det

Non-Deterministic Grammar

- Grammar with common prefix between at least two different production from the same L.H.S (non-terminal in CFG) is called non-deterministic grammar.
  - A -> $\alpha\beta_1 \mid \alpha\beta_2 \mid \alpha\beta_3 \mid \alpha\beta_4$
  - Suppose you have to generate $\alpha\beta_4$
- Such grammar requires backtracking due to confusion which is actually a costly process.
- After tracing th first input 'α', there are 4 available options. So the confusion is to go with β_1 , β_2 , β_3 , β_4
- Due to this confusion, Top down parser don't prefer to parse Non-determinestic gramar.

Checking Non-deterministic Grammar

- **EX1:** Check if the grammar is non-deterministic or not

  S-> aSb | aA | b

  A -> aB | a

  B -> b

Ans: Yes.

- **EX2:** Check if the grammar is non-deterministic or not

  S-> aSb | bSaA | ε

Ans: No

- **EX3:** Check if the grammar is non-deterministic or not

  S-> Aab | BA

  A -> a | b

  B -> d | e

Ans: No.

Ex4: Check if the grammar is non-deterministic or not

A -> a β_1

B -> a β_2

Ans: No.

Deterministic grammar

- Without any common prefix in any of the different productions from same L.H.S (non-terminal in CFG).
- Examples:
  - EX2, EX3, EX4

Left factoring

Formulla

A -> $\alpha\beta_1 \mid \alpha\beta_2 \mid \alpha\beta_3 \mid \alpha\beta_4$

Now converting it to deterministic

A -> α A′

A′ -> $\beta_1 \mid \beta_2 \mid \beta_3 \mid \beta_4$

- **EX1:**

  S -> iEtS | iEtSeS | a

  E -> b

Convert it to deterministic grammar

Solution:

S -> iEtS S′ | a

S′ -> ϵ | eS

E -> b

Left factoring

- **EX2:**

  S -> aSSbS | aSaSb | abb | b

Convert it to deterministic grammar

Solution:

S -> aS′ | b

S′ -> SSbS | SaSb | bb

S′ -> S S″ | bb

S″ -> SbS | aSb

- So the final result is:

  S -> aS′ | b

  S′ -> S S″ | bb

  S″ -> SbS | aSb

Left factoring

- **EX4:**

  S -> a | ab | abc | abcd

Convert it to deterministic grammar

Solution:

S -> aS′

S′ -> ε | b | bc | bcd

S′ -> ε | bS″

S″ -> ε | c | cd

- S″ -> ε | c S‴

  S‴ -> ε | d

So, the final result is

S -> aS′

S′ -> ε | bS″

S″ -> ε | c S‴

S‴ -> ε | d

Aspect	Top-Down Parser	Bottom-Up Parser
Starting Point	Begins parsing from the **start symbol (root)**.	Begins parsing from the **input tokens (leaves)**.
Parsing Direction	Expands productions **from top to bottom** of the parse tree.	Reduces input **from bottom to top** into the start symbol.
Approach	**Prediction-based**: chooses which production to apply.	**Reduction-based**: combines tokens into higher-level rules.
Builds Tree	Builds the parse tree **from root to leaves**.	Builds the parse tree **from leaves to root**.
Common Techniques	Recursive Descent, LL Parsers	Shift-Reduce, LR Parsers
Backtracking	Often needs backtracking (unless LL(1))	Generally no backtracking needed
Ease of Implementation	Easier to write manually	More complex; usually generated by parser tools
Efficiency	Less efficient for ambiguous grammars	More efficient and powerful; handles a wider class of grammars
Grammar Type Supported	Mostly works for **LL grammars**	Works for **LR grammars (more general)**

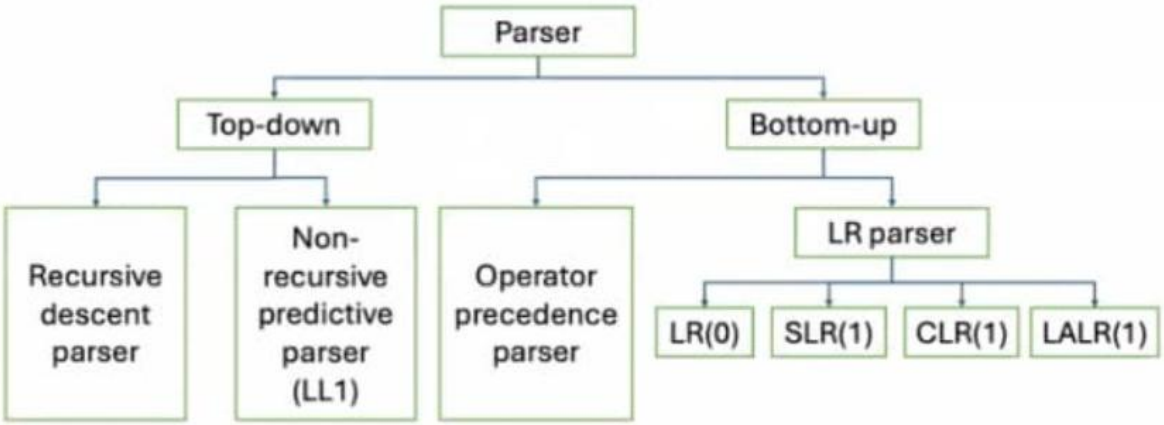
Regular Grammar vs Context-free Grammar

Regular Grammar	Context-free Grammar
It is not suitable for parsing.	Parsing is suitable
Type-3 Grammar	Type-2 Grammar
It is a subset of CFG	Every CFG may not be RG.
Syntax and any programming language can't be represented completely by RG	Syntax and any programming language can be represented completely by CFG

Types of SDD

S - Attributed	L - Attributed
A SDD that uses only synthesized attribute is called S-Attributed SDD. **Ex:** A → BCD A.i = B.i A.i = C.i A.i = D.i	A SDD that uses both synthesized and inherited attributes is called as L-attributed SDD but each inherited attribute is restricted to inherit from parent or left siblings only. **Ex:** A → BCD C.i = A.i C.i = B.i C.i = D.i (Not allowed)
Semantic actions are always placed at right end of the production (Postfix SDD)	Semantic action are placed anywhere on the R.H.S of the production
Use bottom up parsing	Use top down parsing
It is a subset of L attribute	Every L attribute is not a S attribute

Types of Parser



Check The Type of Attributes

- A → BC { B.i = A.i, C.i = B.i, A.i = C.i }
  - Ans: L attribute, Not S attribute
- A → QR { R.i = A.i, Q.i = R.i, A.i = Q.i}
  - Ans: Not L attribute, Not S attribute
- A → PQ { A.i = P.i, A.i = Q.i}
  - Ans: L attribute, S attribute

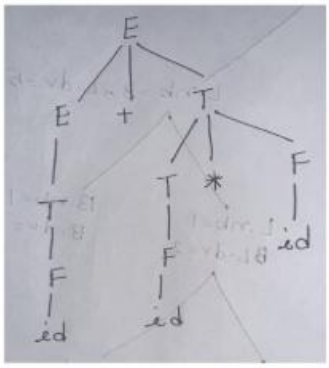
SDD to Syntax Tree

Production	Semantic Rules
E → E + T	{ E.nptr = mknode(E.nptr, +, T.nptr) }
E → T	{ E.nptr = T.nptr }
T → T * F	{ T.nptr = mknode(T.nptr, *, F.nptr) }
T → F	{ T.nptr = F.nptr }
F → id	{ F.nptr = mknode(Null, id, Null) }

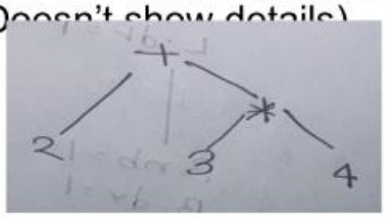
- In order to understand syntax tree let's derive a parse tree for this string '2+3*4'.

SDD to Syntax Tree

- This is parse tree. It's another name is concrete syntax tree.

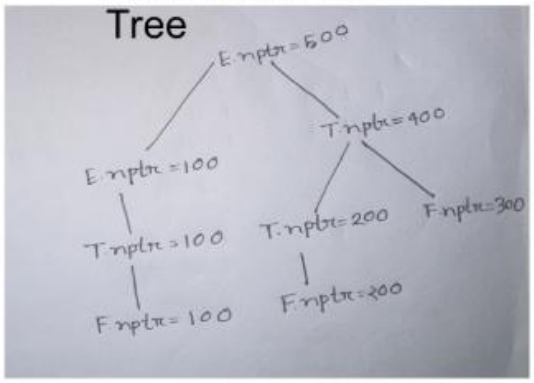


- We can use less no of nodes and represent the same string.
- Then it will be called syntax tree or abstract syntax tree. (Doesn't show details)

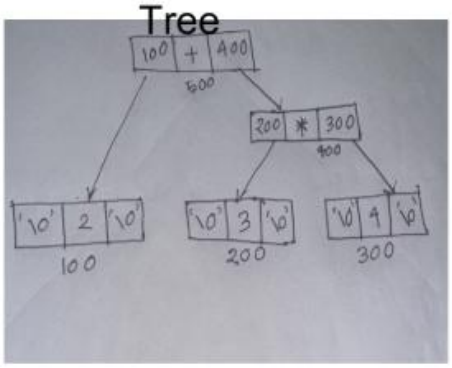


SDD to Syntax Tree

Annotated Parse Tree



Syntax Tree



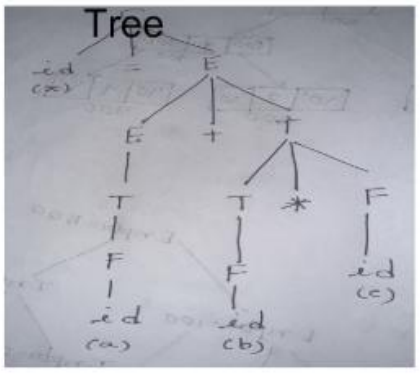
SDD to Generate Three Address Code

Production	Semantic Rules
$S \rightarrow id = E$	{ gen (id.name = E.place); }
$E \rightarrow E + T$	{ E.place = new temp(); gen(E.place = E.place + T.place); }
$E \rightarrow T$	{ E.place = T.place }
$T \rightarrow T * F$	{ T.place = new temp(); gen(T.place = T.place * F.place); }
$T \rightarrow F$	{ T.place = F.place }
$F \rightarrow id$	{ F.place = id.name }

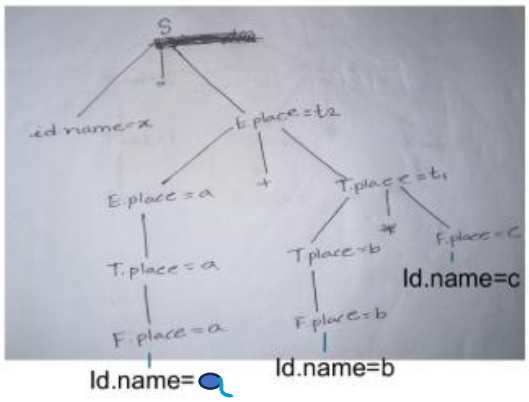
SDD to Generate Three Address Code

- In order to understand syntax tree let's derive a parse tree for this string 'x=a+b*c'.

Parse Tree



Annotated Parse



Output

$t_1 = b*c$

$t_2 = a+t_1$

$x = t_2$

Introduction

- SDD = Grammar + Semantic Rules
- Semantic means the parse tree which is generated is meaningful.
- Meaningful means the operations which are performed maintained the rules.
- Applications:
  - Executing arithmetic expressions
  - Conversion from binary to decimal
  - Creating syntax tree
  - Type checking etc.

Productions and Semantic Actions (Example1)

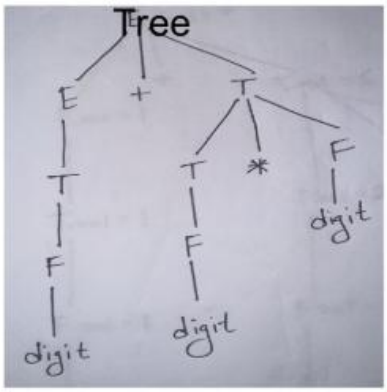
Production	Semantic Rules
$E \rightarrow E + T$	{ E.val = E.val + T.val }
$E \rightarrow T$	{ E.val = T.val }
$T \rightarrow T * F$	{ T.val = T.val * F.val }
$T \rightarrow F$	{ T.val = F.val }
$F \rightarrow digit$	{ F.val = digit.lval }

- Attributes are assoiciated with grammar symbols and semantic rules are associated with productions.
- Attributes value may be number, strings, reference, datatype etc.

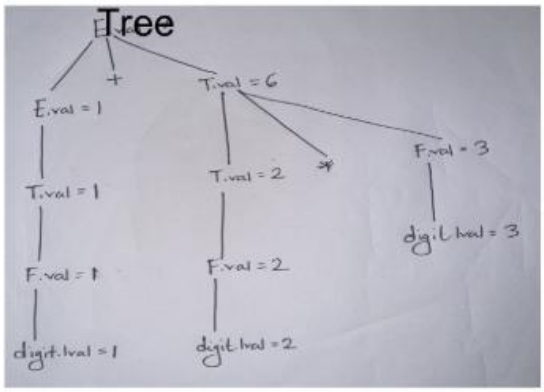
Example1 (Cont'd)

- Given the string 1 + 2 * 3 . Now by using annotated (Symantically Verified) parse tree show the output

Parse Tree



Annotated Parse Tree



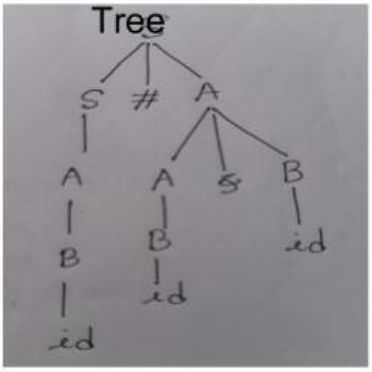
Example2

Production	Semantic Rules
S → S # A | A	{ S.val = S.val * A.val, S.val = A.val }
A → A & B | B	{ A.val = A.val + B.val, A.val = B.val }
B → id	{ B.val = id.lval }

Given the string 5 # 3 & 4, Now by using annotated parse tree show the output

Example2 (Cont'd)

Parse Tree



Annotated Parse